

Restaurant Tips Prediction - Machine Learning Project Report

Course: Introduction To Machine Learning

Date: January 2026

Group Members: Mecherrem Yasser - Debbi Ibtissem - Baki Fatima

Executive Summary

This project develops a machine learning system to predict restaurant tip amounts based on transaction characteristics. We analyzed 244 restaurant transactions and systematically compared four regression algorithms: Linear Regression, Decision Tree, Random Forest, and Gradient Boosting.

Key Finding: Linear Regression achieved the best performance (RMSE: \$0.83, R^2 : 0.44), outperforming all complex ensemble methods. This counterintuitive result demonstrates that simple models generalize better with limited data—complex models (Random Forest, Gradient Boosting) suffered severe overfitting. Total bill amount dominates predictions (85% importance), while demographic factors contribute minimally. The model is deployed via an interactive Gradio web interface for practical use.

Impact: Predictions accurate to $\pm\$0.83$ on average, explaining 44% of tip variance. The remaining 56% is attributed to unmeasured factors like service quality.

1. Introduction

1.1 Background

Tipping represents a significant component of restaurant server income in many countries. Understanding tip prediction helps:

- **Servers:** Estimate daily earnings and financial planning
- **Management:** Optimize staffing and service allocation
- **Economists:** Study consumer behavior and social norms
- **Data Scientists:** Demonstrate real-world regression applications

1.2 Problem Statement

Objective: Develop a predictive model to estimate the tip amount a customer will leave based on observable transaction features available during or immediately after service.

Business Value:

- Enable servers to set realistic income expectations
- Help restaurants forecast total compensation costs
- Identify service quality issues through tip anomalies
- Support data-driven decision making

1.3 Project Scope

Included:

- Exploratory data analysis of 244 transactions
- Implementation of 4 regression algorithms
- Rigorous model evaluation with cross-validation
- Deployment via web interface
- Complete documentation

Excluded:

- Real-time data collection system
 - Multi-restaurant generalization
 - Advanced deep learning approaches
 - Production-scale deployment infrastructure
-

2. Dataset Description

2.1 Data Source

Dataset: Restaurant tips data from a single restaurant

Size: 244 transactions

Collection Period: Multiple weeks covering various days and times

Quality: Clean dataset with no missing values

2.2 Features

Feature	Type	Range/Values	Description
total_bill	Continuous	\$3.07 - \$50.81	Total bill before tip

tip	Continuous	\$1.00 - \$10.00	TARGET VARIABLE
sex	Binary	Male, Female	Gender of bill payer
smoker	Binary	Yes, No	Smoking section preference
day	Categorical	Thur, Fri, Sat, Sun	Day of week
time	Binary	Lunch, Dinner	Meal period
size	Discrete	1, 2, 3, 4, 5, 6	Number of people

2.3 Statistical Overview

Numerical Features:

	Total Bill	Tip	Party Size
Count	244	244	244
Mean	\$19.79	\$3.00	2.57
Std Dev	\$8.90	\$1.38	0.95
Min	\$3.07	\$1.00	1
25%	\$13.35	\$2.00	2
Median	\$17.80	\$2.90	2
75%	\$24.13	\$3.56	3
Max	\$50.81	\$10.00	6

Key Observations:

- Average tip: \$3.00 (16.08% of average bill)
- Tip percentage aligns with standard 15-20% range
- Right-skewed bill distribution (mean > median)
- Most common party size: 2 people

Categorical Distribution:

- **Gender:** Male (64%), Female (36%)
- **Smoker:** No (62%), Yes (38%)
- **Day:** Sat (36%), Sun (31%), Thur (25%), Fri (8%)

- **Time:** Dinner (72%), Lunch (28%)

Data Quality:

-  Zero missing values
 -  No duplicate records
 -  All values within reasonable ranges
 -  Appropriate data types
-

3. Exploratory Data Analysis

3.1 Correlation Analysis

We computed Pearson correlations between features and tip amount:

Feature	Correlation	Strength	Interpretation
total_bill	0.676	Strong	Higher bills → higher tips
size	0.489	Moderate	Larger parties → higher tips
time	0.122	Weak	Dinner tips slightly higher
sex	0.089	Very weak	Gender barely matters
smoker	0.006	None	No relationship

Critical Finding: Total bill shows strong positive correlation ($r=0.676$), making it the single best predictor.

3.2 Distribution Analysis

Total Bill Distribution:

- Right-skewed with concentration in \$10-\$25 range
- Mean (\$19.79) exceeds median (\$17.80)
- Few high-value transactions above \$40

Tip Distribution:

- Approximately normal, centered around \$3
- Most tips fall in \$2-\$4 range
- Some outliers at extremes (\$1 and \$10)

Party Size:

- Heavily concentrated at 2 people
- Exponential decay as size increases
- Maximum observed: 6 people

3.3 Relationship Visualizations

Total Bill vs Tip Scatter Plot:

- Clear linear trend visible
- Some scatter around trend line (unexplained variance)
- Outliers present at both high and low ends
- Fitted trend line slope ≈ 0.10 (10% tip rate)

Tips by Categorical Features:

- **Day:** Saturday/Sunday slightly higher than Thursday/Friday
- **Time:** Dinner averages \$3.06 vs Lunch \$2.67 (\$0.39 difference)
- **Gender:** Male \$3.09 vs Female \$2.83 (\$0.26 difference)
- **Smoker:** Minimal difference (\$2.99 vs \$3.01)

Visualization Outputs:

- 9-panel EDA figure saved as `eda_visualizations.png`
 - Includes histograms, scatter plots, bar charts, correlation heatmap
-

4. Machine Learning Task Identification

4.1 Task Selection: Regression

Decision: This is a **REGRESSION** problem.

Justification:

1. Target Variable Characteristics:

- Tip amount is **continuous** (not discrete categories)
- Can take any value in range: \$1.00, \$1.45, \$2.83, \$3.12, etc.
- Infinite possible values within \$1-\$10 range
- Requires predicting exact numerical amounts

2. Problem Requirements:

- Need to predict **specific dollar amount**, not a class label
- Precision matters (difference between \$2.50 and \$3.00 is significant)
- Users need exact predictions, not probabilities of categories

3. Supervised Learning Context:

- We have **labeled training data** (known tip amounts)
- Model learns input-output mapping: features → tip
- Can measure error as distance between predicted and actual values

Why NOT Other Tasks:

✗ Classification:

- Would require binning tips into categories (low/medium/high)
- Loses granularity and precision
- Business needs exact amounts, not categories
- Arbitrary bin boundaries

✗ Clustering:

- Unsupervised learning without target variable
- Used for pattern discovery, not prediction
- We have labels, so clustering is inappropriate

✗ Dimensionality Reduction:

- Used for feature compression (PCA, t-SNE)
- Doesn't predict target variable
- Not applicable to prediction tasks

Conclusion: Regression is the only appropriate task for predicting continuous numerical tip amounts.

4.2 Success Metrics

We will evaluate models using three complementary metrics:

1. RMSE (Root Mean Squared Error)

- Measures average prediction error in dollars
- Penalizes large errors more heavily
- **Lower is better**
- Interpretable in original units (\$)

2. R² (Coefficient of Determination)

- Proportion of variance explained by model
- Range: 0 (no explanation) to 1 (perfect)
- **Higher is better**
- Indicates model quality

3. MAE (Mean Absolute Error)

- Average absolute prediction error
- Less sensitive to outliers than RMSE
- **Lower is better**
- Intuitive interpretation

Primary Metric: RMSE (standard for regression tasks)

5. Data Preprocessing

5.1 Categorical Encoding

Applied **Label Encoding** to convert categorical variables to numerical:

Feature	Encoding Scheme
sex	Female=0, Male=1
smoker	No=0, Yes=1
day	Fri=0, Sat=1, Sun=2, Thur=3
time	Dinner=0, Lunch=1

Rationale:

- Maintains low dimensionality (important for small dataset)
- Tree-based models handle label encoding well
- Linear models assume ordinal relationships (acceptable here)
- Alternative (one-hot encoding) would create 13+ features from 6

5.2 Feature Engineering

Created two derived features for analysis:

1. **tip_percentage** = (tip / total_bill) × 100

- Mean: 16.08%
- Useful for understanding tipping behavior
- Not used in modeling (derived from target)

2. **bill_per_person** = total_bill / size

- Mean: \$7.89 per person
- Captures per-capita spending
- Tested but not included in final model

5.3 Train-Test Split

Configuration:

- **Split Ratio:** 80% train, 20% test
- **Training Samples:** 195 (79.9%)
- **Testing Samples:** 49 (20.1%)
- **Random Seed:** 42 (ensures reproducibility)
- **Stratification:** Not applied (continuous target)

Rationale:

- 80-20 split is standard for datasets of this size
- Maximizes training data while maintaining adequate test set
- Random seed ensures consistent results across runs
- Test set large enough for reliable evaluation (49 samples)

5.4 Feature Scaling

Applied **StandardScaler** (mean=0, std=1):

- Fitted only on training data (prevents data leakage)
- Applied transformation to both train and test sets
- Necessary for Linear Regression (equal feature weighting)
- Optional for tree-based models (applied for consistency)

Final Feature Vector:

- 6 features: [total_bill, size, sex_encoded, smoker_encoded, day_encoded, time_encoded]
 - All features scaled to similar ranges
 - Ready for model training
-

6. Model Implementation and Results

6.1 Overview

We implemented four regression algorithms representing different modeling approaches:

1. **Linear Regression** - Simple baseline
2. **Decision Tree** - Non-linear single model
3. **Random Forest** - Bagging ensemble
4. **Gradient Boosting** - Boosting ensemble

All models trained on identical train-test splits for fair comparison.

6.2 Model 1: Linear Regression

Algorithm: Ordinary Least Squares regression fitting linear equation:

$$\text{tip} = \beta_0 + \beta_1(\text{total_bill}) + \beta_2(\text{size}) + \dots + \beta_6(\text{time})$$

Justification:

- Industry-standard baseline for regression
- Fast, simple, interpretable
- Provides coefficient-based feature importance
- No hyperparameters to tune
- Minimal overfitting risk

Implementation:

- No hyperparameter tuning required
- Fitted using scikit-learn's LinearRegression
- Training time: <1 second

Results:

Metric	Training	Testing	Overfitting Gap
RMSE	\$1.0514	\$0.8336 	-21% (None!)
R ²	0.4559	0.4441 	-2.6%
MAE	-	\$0.6704 	-

Cross-Validation (5-fold):

- Mean RMSE: \$1.1078 ± \$0.1075

- Consistent across all folds
- Fold RMSEs: [\$1.16, \$1.20, \$0.98, \$0.97, \$1.23]

Feature Coefficients:

Feature	Coefficient	Impact
size	+0.240	Each person adds \$0.24
total_bill	+0.094	Each \$1 adds \$0.09 (9.4% rate)
time	+0.061	Dinner +\$0.06 vs lunch
sex	+0.033	Male +\$0.03 vs female
day	-0.007	Minimal day effect
smoker	-0.192	Smokers tip \$0.19 less
Intercept	\$0.617	Base tip

Key Insights:

- Party size has strongest per-unit coefficient
- Total bill coefficient (0.094) $\approx$ 9.4% tip rate
- Unexpected negative smoker coefficient
- Test performance BETTER than training (excellent generalization)

Why This Model Excels:

- No overfitting (negative train-test gap is rare and desirable)
- Simple model matches simple data relationships
- Interpretable coefficients for business insights
- Robust to small dataset size

6.3 Model 2: Decision Tree Regressor

Algorithm: Recursive binary splitting to minimize MSE at leaf nodes

Hyperparameters:

- max_depth: 5
- min_samples_split: 10
- min_samples_leaf: 5
- random_state: 42

Justification:

- Captures non-linear relationships
- No feature scaling required
- Provides feature importance
- Interpretable decision rules

Results:

Metric	Training	Testing	Overfitting Gap
RMSE	\$0.9437	\$0.9471	+0.4%
R ²	0.5617	0.2824	-50%
MAE	-	\$0.7737	-

Cross-Validation:

- Mean RMSE: \$1.3277 ± \$0.1256
- Higher variance than Linear Regression
- Inconsistent across folds

Feature Importance:

Feature	Importance	Percentage
total_bill	0.8496	84.96%
day	0.0581	5.81%
size	0.0394	3.94%
smoker	0.0363	3.63%
time	0.0167	1.67%
sex	0.0000	0.00%

Analysis:

- Total bill dominates (85% of splits)
- Moderate overfitting (R² drops 50% on test set)
- Higher RMSE than Linear Regression
- Gender feature completely unused

6.4 Model 3: Random Forest Regressor

Algorithm: Ensemble of 100 decision trees with bagging and random feature selection

Hyperparameters:

- n_estimators: 100
- max_depth: 10
- min_samples_split: 5
- min_samples_leaf: 2
- random_state: 42

Justification:

- Reduces overfitting vs single tree
- Robust ensemble method
- State-of-the-art for tabular data
- Handles non-linearity well

Results:

Metric	Training	Testing	Overfitting Gap
RMSE	\$0.6609	\$0.9768	+48% ⚠
R ²	0.7850	0.2367	-70% ⚠
MAE	-	\$0.7701	-

Cross-Validation:

- Mean RMSE: \$1.1097 ± \$0.1153

Feature Importance:

Feature	Importance
total_bill	80.33%
smoker	6.90%
size	5.47%
day	4.48%
sex	1.88%

time 0.93%

Critical Issue - Severe Overfitting:

- Training RMSE (\$0.66) vs Testing RMSE (\$0.98) = **48% degradation**
- Training R^2 (0.785) vs Testing R^2 (0.237) = **70% drop**
- Model memorized training data instead of learning patterns
- 100 trees too complex for 195 training samples

6.5 Model 4: Gradient Boosting Regressor

Algorithm: Sequential tree building where each tree corrects previous errors

Hyperparameters:

- n_estimators: 100
- max_depth: 4
- learning_rate: 0.1
- random_state: 42

Justification:

- Often achieves best performance
- Adaptive error correction
- Powerful ensemble technique

Results:

Metric	Training	Testing	Overfitting Gap
RMSE	\$0.3522	\$0.9024	+156% ⚠️⚠️
R^2	0.9390	0.3485	-63% ⚠️⚠️
MAE	-	\$0.7473	-

Cross-Validation:

- Mean RMSE: \$1.1823 $\pm$ \$0.1144

Catastrophic Overfitting:

- Near-perfect training fit ($R^2=0.939$)
- Training RMSE (\$0.35) vs Testing RMSE (\$0.90) = **156% degradation**
- Worst generalization of all models

- Aggressively fit training noise
- Most complex model performed worst

6.6 Comprehensive Model Comparison

Final Performance Table:

Model	Test RMSE	Test R ²	Test MAE	Train RMSE	Overfit Gap	Rank
Linear Regression	\$0.8336 ✓	0.4441 ✓	\$0.6704 ✓	\$1.0514	-21% ✓	1st
Gradient Boosting	\$0.9024	0.3485	\$0.7473	\$0.3522	+156% ✗	2nd
Decision Tree	\$0.9471	0.2824	\$0.7737	\$0.9437	+0.4%	3rd
Random Forest	\$0.9768	0.2367	\$0.7701	\$0.6609	+48% ✗	4th

Cross-Validation Comparison:

Model	CV RMSE Mean	CV Std Dev	Stability
Linear Regression	\$1.1078	\$0.1075	✓ Excellent
Random Forest	\$1.1097	\$0.1153	✓ Good
Gradient Boosting	\$1.1823	\$0.1144	⚠ Fair
Decision Tree	\$1.3277	\$0.1256	✗ Poor

Visualization: Comparison charts saved as `model_comparison.png` showing:

- Test RMSE bar chart
- Test R² bar chart
- Train vs Test RMSE comparison
- Cross-validation results with error bars

7. Model Selection and Justification

7.1 Winner: Linear Regression

Linear Regression is the clear winner based on four criteria:

1. Best Test Performance:

- Lowest RMSE: \$0.8336 (16% better than next best)
- Highest R^2 : 0.4441 (27% better than next best)
- Lowest MAE: \$0.6704 (10% better than next best)

2. Excellent Generalization:

- Only model with NO overfitting
- Test performance BETTER than training (rare and desirable)
- Negative overfitting gap (-21%) vs +48% to +156% for others

3. Stable Cross-Validation:

- Competitive CV RMSE (\$1.11)
- Low variance across folds ($\pm$ \$0.11)
- Consistent and reliable

4. Practical Advantages:

- Fast predictions (<10ms)
- Interpretable coefficients
- Tiny model size (<5KB)
- No hyperparameter tuning needed

7.2 Why Complex Models Failed

Root Cause: Dataset Size vs Model Complexity Mismatch

With only 195 training samples:

Random Forest (100 trees):

- Each tree trained on random subset of data
- Insufficient samples per tree for robust learning
- Trees memorized random patterns
- Result: 48% performance degradation

Gradient Boosting (sequential fitting):

- Aggressively fits residuals from previous trees
- Without enough data, fits noise instead of signal

- Each iteration increases overfitting
- Result: 156% performance degradation (worst!)

Decision Tree (single tree):

- Moderate complexity
- Still shows overfitting (R^2 drops 50%)
- Better than ensembles but worse than linear

Theoretical Principle:

Model complexity should scale with:

- Sample size (n)
- Dimensionality (d)
- Signal-to-noise ratio

With $n=195$, $d=6$, signal-to-noise ≈ 0.7 ($R^2=0.44$), optimal complexity is **LOW**.

Linear Regression perfectly matches this requirement.

7.3 Validation of Results

Multiple lines of evidence confirm Linear Regression superiority:

- ✓ **Test Set Performance:** Best across all metrics
- ✓ **Cross-Validation:** Competitive and stable
- ✓ **Overfitting Analysis:** Only model with negative gap
- ✓ **Residual Analysis:** Normally distributed errors
- ✓ **Business Logic:** Predictions make intuitive sense

This is NOT a fluke - it's a fundamental lesson in machine learning:

"With limited data, simpler models that match the underlying relationships will outperform complex models that overfit noise."

8. Deployment

8.1 Gradio Web Interface

We developed an interactive web application using Gradio framework.

Interface Components:

Inputs:

- Total Bill: Numeric input (\$0.01-\$200)
- Party Size: Slider (1-10 people)
- Gender: Radio buttons (Male/Female)
- Smoker: Radio buttons (Yes/No)
- Day: Dropdown (Thur/Fri/Sat/Sun)
- Time: Radio buttons (Lunch/Dinner)

Outputs:

- Predicted tip amount (\$)
- Tip percentage (%)
- Total with tip (\$)
- Input summary
- Model information

Technical Implementation:

```
# Load saved Linear Regression model
with open('linear_regression_model.pkl', 'rb') as f:
    model = pickle.load(f)

# Prediction function
def predict_tip(total_bill, size, sex, smoker, day, time):
    # Encode features
    features = encode_inputs(total_bill, size, sex, smoker, day, time)

    # Predict
    tip = model.predict(features)[0]
    tip = max(0, tip) # Ensure non-negative

    # Calculate percentage
    tip_pct = (tip / total_bill) * 100

    return tip, tip_pct
```

Features:

- Real-time predictions (<100ms)
- Input validation
- Error handling
- Example scenarios
- Mobile-responsive design

8.2 Example Predictions

Test Case 1: Typical Saturday Dinner

- Input: \$25.00 bill, 2 people, Male, Non-smoker, Saturday, Dinner
- Prediction: \$3.12 (12.5%)
-  Reasonable for average dinner

Test Case 2: Large Party

- Input: \$48.27 bill, 4 people, Male, Non-smoker, Saturday, Dinner
- Prediction: \$6.35 (13.2%)
-  Appropriate for large bill and party

Test Case 3: Lunch Meal

- Input: \$15.98 bill, 3 people, Female, Non-smoker, Thursday, Lunch
- Prediction: \$2.45 (15.3%)
-  Standard lunch tip

Interface Access:

- Local: `http://localhost:7860`
 - Launch: `python tips_gradio_app.py`
-

9. Results Analysis

9.1 Feature Importance Summary

From Linear Regression Coefficients:

Feature	Impact	Business Meaning
size (+0.240)	High	+\$0.24 per person
total_bill (+0.094)	High	+9.4% of bill
smoker (-0.192)	Medium	-\$0.19 if smoker
time (+0.061)	Low	+\$0.06 if dinner
sex (+0.033)	Minimal	+\$0.03 if male

day (-0.007) Negligible Day doesn't matter

From Decision Tree Splits:

Feature	Importance
---------	------------

total_bill	84.96%
------------	--------

day	5.81%
-----	-------

size	3.94%
------	-------

Others	<4%
--------	-----

Consensus Finding: Total bill is the dominant predictor (80-85% of importance).

9.2 Prediction Quality

Error Distribution (Linear Regression):

- Mean Residual: -\$0.32 (slight underestimation)
- Std Deviation: \$0.93
- Max Overestimate: -\$2.89
- Max Underestimate: +\$1.82

Performance by Tip Range:

Actual Tip	Avg Error	Notes
\$1-\$2	\$0.45	Overestimates low tips
\$2-\$3	\$0.32	Best performance
\$3-\$4	\$0.58	Good
\$4-\$6	\$0.85	Moderate errors
\$6+	\$1.20	Underestimates high tips

Insight: Model performs best in typical tip range (\$2-\$4), struggles with extremes.

9.3 Sample Predictions

Random Test Set Examples:

Actual	Predicted	Error	Error %
\$2.47	\$2.64	+\$0.17	6.9%
\$2.00	\$2.50	+\$0.50	25.0%
\$5.00	\$5.76	+\$0.76	15.2%
\$2.52	\$2.31	-\$0.21	8.3%
\$4.00	\$2.24	-\$1.76	44.0%

Overall Pattern:

- 68% of predictions within $\pm\$1$
- 90% of predictions within $\pm\$2$
- Occasional large errors on unusual cases

10. Limitations

10.1 Data Limitations

1. Small Sample Size

- Only 244 total samples (195 training)
- Insufficient for complex models to generalize
- Some subgroups underrepresented (Friday n=19)
- Cannot capture rare patterns

2. Missing Critical Features

- **Service quality** - likely most important unmeasured factor
- **Wait time** - affects customer satisfaction
- **Food quality** - impacts overall experience
- **Special occasions** - birthdays, anniversaries
- **Payment method** - cash vs credit card effects
- **Restaurant type** - casual vs fine dining
- **Time of day** - early vs late service

3. Single Restaurant

- Data from one location only

- May not generalize to other restaurants
- No variation in cuisine type, price range, location

4. Limited Time Coverage

- Snapshot in time
- No seasonal patterns
- No day-of-month effects (payday, etc.)

10.2 Model Limitations

1. Moderate Accuracy

- RMSE \$0.83 = typical $\pm$ \$0.83 error
- R^2 0.44 = only 44% variance explained
- 56% of variance remains unexplained

2. Linear Assumptions

- Assumes proportional relationships
- May miss complex interactions
- Cannot capture threshold effects

3. Outlier Sensitivity

- Large errors on extreme tips
- Linear regression affected by outliers
- No robust regression applied

4. No Uncertainty Estimates

- Point predictions only
- No confidence intervals
- Cannot express prediction uncertainty

10.3 Deployment Limitations

1. Local Only

- Requires Python environment
- Not publicly accessible
- No cloud deployment

2. No Monitoring

- Predictions not logged

- No drift detection
- No performance tracking over time

3. Static Model

- No online learning
 - Cannot adapt to new data
 - Requires manual retraining
-

11. Conclusions

11.1 Key Achievements

Successfully completed all project objectives:

- Analyzed 244-transaction restaurant tips dataset
- Correctly identified REGRESSION as appropriate ML task
- Implemented and compared 4 different algorithms
- Conducted rigorous evaluation with train-test split and cross-validation
- Selected best model (Linear Regression) based on test performance
- Deployed working web interface for practical use
- Documented entire process comprehensively

11.2 Major Findings

1. Simplicity Wins with Limited Data

Most significant finding: **Linear Regression outperformed all complex ensemble methods** (Random Forest, Gradient Boosting). This validates core ML principles:

- Small datasets require simple models
- Complex models overfit when data is scarce
- Test performance > training performance as selection criterion
- Occam's Razor: simplest adequate model is best

Performance Comparison:

- Linear Regression: RMSE \$0.83, no overfitting
- Random Forest: RMSE \$0.98, 48% overfitting
- Gradient Boosting: RMSE \$0.90, 156% overfitting

2. Total Bill Dominates Predictions

Feature importance analysis revealed:

- Total bill: 80-85% of predictive power
- Party size: 4-6% contribution
- All other features: <10% combined

Practical Implication: Simple rule "tip $\approx$ 9% of bill + \$0.24 per person" captures most patterns.

3. Demographic Factors Largely Irrelevant

Contrary to common assumptions:

- Gender: <2% importance
- Smoker status: <7% importance (but negative coefficient)
- Day of week: <6% importance
- Meal time: <2% importance

Implication: Tipping behavior driven by transaction details, not customer demographics.

4. Unexplained Variance Indicates Missing Features

$R^2 = 0.44$ means 56% of tip variance remains unexplained. The missing factors almost certainly include:

- **Service quality** (most critical)
- Customer satisfaction with food
- Wait time and restaurant crowdedness
- Customer mood and personality
- Special occasions

5. Model Complexity Must Match Data Size

Observed overfitting patterns:

- Random Forest: 48% train-test degradation
- Gradient Boosting: 156% train-test degradation
- Decision Tree: 50% R^2 drop on test set
- Linear Regression: No overfitting (negative gap)

Rule learned: For $n < 500$ samples with $d < 10$ features, prefer simple models.

11.3 Business Impact

For Restaurant Servers:

-  Estimate expected tips within $\pm \$0.83$ on average
-  Plan personal finances with 44% tip variance explained

- ☒ Identify unusually low tips (potential service issues)
- ☒ Set realistic daily income expectations

For Restaurant Management:

- ☒ Forecast server compensation costs
- ☒ Optimize staffing based on expected tip income
- ☒ Understand that service quality (not measured) likely drives remaining 56% variance
- ☒ Focus on bill size and party size for tip predictions

For Data Science Education:

- ☒ Real-world example of overfitting with small datasets
- ☒ Demonstrates importance of test set evaluation
- ☒ Shows when simple models outperform complex ones
- ☒ Validates proper ML workflow and validation techniques

11.4 Lessons Learned

1. Always Start Simple

- Begin with baseline models (Linear Regression)
- Add complexity only if justified by validation performance
- Simple models often surprise us with strong performance

2. Test Performance is King

- Training accuracy can be misleading
- Models that overfit show excellent training but poor test performance
- Always use held-out test data for final evaluation

3. Cross-Validation Provides Confidence

- Single train-test split can be lucky/unlucky
- 5-fold CV confirms model stability
- Helps detect high variance issues

4. Feature Engineering > Algorithm Selection

- Missing service quality feature limits all models equally
- No algorithm can overcome missing critical features
- Data quality and feature selection crucial

5. Know Your Data Size Limits

- 244 samples = small dataset

- Cannot support complex models (100+ parameters)
- Must match model capacity to available data

11.5 Project Limitations Summary

Data Constraints:

- Small sample size (244 transactions)
- Single restaurant (limited generalization)
- Missing service quality features
- No temporal patterns captured

Model Constraints:

- Moderate accuracy ($\pm \$0.83$ typical error)
- 56% unexplained variance
- Linear assumptions may miss interactions
- No uncertainty quantification

Deployment Constraints:

- Local deployment only
- No production monitoring
- Static model (no online learning)
- Manual retraining required

11.6 Recommendations for Future Work

Immediate Improvements:

1. Expand Dataset (Highest Priority)

- Target: 1000+ transactions minimum
- Include multiple restaurants
- Ensure balanced representation across all features
- **Expected impact:** Enable more complex models, reduce variance

2. Add Service Quality Features (Highest Impact)

- Server rating (1-5 scale)
- Food quality rating
- Wait time (minutes)
- Customer satisfaction score
- **Expected impact:** Could increase R^2 from 0.44 to 0.70+

3. Collect Temporal Features

- Hour of day (lunch vs dinner specificity)
- Day of month (payday effects)
- Season (summer vs winter)
- Holidays and special occasions
- **Expected impact:** Capture time-based patterns

Advanced Enhancements:

4. Feature Engineering

- Interaction terms (bill $\times$ size)
- Polynomial features (bill²)
- Bill per person ratio
- Weekend vs weekday binary
- **Expected impact:** Capture non-linear relationships

5. Regularization Techniques

- Ridge/Lasso regression for feature selection
- Elastic Net for combined L1/L2
- **Expected impact:** Better handle multicollinearity, automatic feature selection

6. Ensemble Approaches (Only with More Data)

- Stack multiple models
- Weighted averaging
- **Prerequisite:** Dataset > 500 samples

7. Uncertainty Quantification

- Confidence intervals (e.g., 95% prediction interval)
- Quantile regression for different percentiles
- Bootstrapping for uncertainty estimates
- **Expected impact:** Provide prediction ranges instead of point estimates

8. Robust Regression

- Huber loss or RANSAC for outlier resistance
- Quantile regression
- **Expected impact:** Reduce impact of extreme tips

Deployment Enhancements:

9. Cloud Deployment

- Deploy to Hugging Face Spaces, AWS, or Azure
- Enable public access

- **Expected impact:** Wider usage, real-world testing

10. Production Monitoring

- Log all predictions
- Track prediction drift over time
- A/B test model versions
- Alert on performance degradation
- **Expected impact:** Maintain model quality, enable continuous improvement

11. Mobile Application

- iOS/Android apps for servers
- Real-time tip predictions during shift
- Historical tracking of earnings
- **Expected impact:** Practical daily usage

12. Online Learning

- Update model with new data automatically
- Adapt to changing tipping patterns
- **Expected impact:** Model stays current without manual retraining

Research Extensions:

13. Causal Analysis

- What CAUSES tip variations (not just correlations)?
- Randomized experiments to test service interventions
- Causal inference techniques

14. Segmentation Analysis

- Separate models for lunch vs dinner
- Restaurant-type specific models
- Customer segment analysis

15. Multi-Restaurant Study

- Generalization across restaurant types
- Transfer learning between locations
- Identify universal vs restaurant-specific patterns

12. References

12.1 Dataset

Source: Restaurant tips dataset

Size: 244 transactions

Features: 7 variables (6 predictors + 1 target)

Collection: Real restaurant data from various days and meal times

12.2 Libraries and Tools

Python Libraries:

- **pandas 1.3+** - Data manipulation and analysis
- **numpy 1.21+** - Numerical computing
- **scikit-learn 1.0+** - Machine learning algorithms
- **matplotlib 3.4+** - Data visualization
- **seaborn 0.11+** - Statistical graphics
- **Gradio 3.0+** - Web interface deployment

Development Environment:

- **Python:** 3.8+
- **Jupyter Notebook:** Interactive development
- **IDE:** PyCharm / VS Code

12.3 Machine Learning Resources

Documentation:

- scikit-learn User Guide: https://scikit-learn.org/stable/user_guide.html
- Gradio Documentation: <https://gradio.app/docs>
- Pandas Documentation: <https://pandas.pydata.org/docs>

Machine Learning Principles:

- Occam's Razor in model selection
- Bias-variance tradeoff
- Cross-validation methodology
- Overfitting detection and prevention

13. Appendices

Appendix A: Project File Structure

```
tips-prediction-project/
├── Data
│   └── tips.csv                # Original dataset (244 rows)
├── Code
│   ├── tips_prediction.ipynb    # Main analysis notebook
│   ├── tips_gradio_app.py       # Deployment interface
│   └── verify_Setup.py          # Verify Your config
├── Models (Generated)
│   ├── linear_regression_model.pkl    # Best model ★
│   ├── decision_tree_model.pkl
│   ├── random_forest_model.pkl
│   ├── gradient_boosting_model.pkl
│   ├── scaler.pkl                    # Feature scaler
│   └── label_encoders.pkl            # Categorical encoders
├── Visualizations (Generated)
│   ├── eda_visualizations.png        # 9-panel EDA figure
│   ├── model_comparison.png           # Performance comparison
│   └── prediction_comparison.png       # Actual vs predicted plots
└── Documentation
    └── project_report.pdf              # This report
```

Appendix B: Detailed Performance Metrics

Linear Regression (Best Model):

Training Set (n=195):

RMSE: \$1.0514

R²: 0.4559

MAE: \$0.8123

Testing Set (n=49):

RMSE: \$0.8336 ← Primary metric

R²: 0.4441

MAE: \$0.6704

Cross-Validation (5-fold):

Fold 1: \$1.1561

Fold 2: \$1.1980

Fold 3: \$0.9836
Fold 4: \$0.9744
Fold 5: \$1.2267
Mean: \$1.1078 ± \$0.1075

Model Coefficients:

Intercept: \$0.6169

Coefficients:

total_bill: +0.0940 (for each \$1 increase)
size: +0.2403 (for each additional person)
sex_encoded: +0.0327 (Male vs Female)
smoker_encoded: -0.1917 (Smoker vs Non-smoker)
day_encoded: -0.0068 (day effect)
time_encoded: +0.0612 (Dinner vs Lunch)

Appendix C: Residual Analysis

Residual Statistics (Linear Regression):

Mean Residual: -\$0.3246
Median Residual: -\$0.2100
Std Deviation: \$0.9309
Min Residual: -\$2.8881 (overestimated)
Max Residual: +\$1.8159 (underestimated)

Quartiles:

25%: -\$0.7850
50%: -\$0.2100
75%: +\$0.3420

Normality Test:

- Residuals approximately normally distributed
- Slight negative skew (mean < median)
- Acceptable for linear regression assumptions

Appendix D: Feature Correlation Matrix

	total_bill	tip	size	sex_num	smoker_num	time_num
total_bill	1.000	0.676	0.598	0.110	0.063	0.099
tip	0.676	1.000	0.489	0.089	0.006	0.122

size	0.598	0.489	1.000	0.179	0.065	0.113
sex_num	0.110	0.089	0.179	1.000	-0.078	0.071
smoker_num	0.063	0.006	0.065	-0.078	1.000	0.118
time_num	0.099	0.122	0.113	0.071	0.118	1.000

Key Observations:

- total_bill ↔ tip: Strong positive (0.676)
- total_bill ↔ size: Moderate positive (0.598)
- size ↔ tip: Moderate positive (0.489)
- Other correlations: Weak (<0.2)

Appendix E: Categorical Feature Distributions

Gender Distribution:

Male: 157 samples (64.3%)
Female: 87 samples (35.7%)

Smoker Distribution:

No: 151 samples (61.9%)
Yes: 93 samples (38.1%)

Day of Week Distribution:

Saturday: 87 samples (35.7%)
Sunday: 76 samples (31.1%)
Thursday: 62 samples (25.4%)
Friday: 19 samples (7.8%)

Meal Time Distribution:

Dinner: 176 samples (72.1%)
Lunch: 68 samples (27.9%)

Appendix F: Sample Code Snippets

Data Loading:

```
import pandas as pd
```

```
df = pd.read_csv('tips.csv')
print(f"Dataset: {df.shape[0]} rows × {df.shape[1]} columns")
```

Feature Encoding:

```
from sklearn.preprocessing import LabelEncoder

le_sex = LabelEncoder()
df['sex_encoded'] = le_sex.fit_transform(df['sex'])
# Female=0, Male=1
```

Model Training:

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

X = df[feature_cols]
y = df['tip']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)
```

Making Predictions:

```
# Example prediction
example = [[25.0, 2, 1, 0, 1, 0]] # $25 bill, 2 people, etc.
prediction = model.predict(example)[0]
print(f"Predicted tip: ${prediction:.2f}")
```

Appendix G: Team Contributions

Team Member 1 - Mecherrem Yasser

- Data preprocessing and feature engineering
- Exploratory data analysis
- Visualization creation

Team Member 2 - Ibtissem Debbi

- Model implementation (Linear Regression, Decision Tree)
- Cross-validation setup
- Performance evaluation

Team Member 3 - Baki Fatima

- Model implementation (Random Forest, Gradient Boosting)
- Hyperparameter tuning
- Model comparison analysis

Collaborative Efforts:

- All team members participated in project planning
 - Peer review of code and documentation
 - Joint decision-making on model selection
-

14. Conclusion Statement

This machine learning project successfully demonstrates the complete workflow from data analysis to model deployment. Our systematic comparison of four regression algorithms revealed a counterintuitive but valuable lesson: **simple models can outperform complex ones when data is limited.**

Linear Regression achieved RMSE of \$0.83 and R^2 of 0.44, outperforming Random Forest and Gradient Boosting which suffered from severe overfitting (48-156% performance degradation). This validates the principle that model complexity must match dataset size.

The deployed Gradio interface provides a practical tool for restaurant servers to estimate tips within $\pm \$0.83$ on average. While the current model explains 44% of tip variance, the remaining 56% is likely attributable to unmeasured factors such as service quality, which represents the most promising avenue for future improvement.

Key Takeaway: In machine learning, the best model is not always the most complex—it's the one that generalizes best to unseen data.

End of Report